Bounded Incremental algorithm for Dynamic Single-Sink Shortest Path Algorithm

Preliminaries

Incremental algorithm - algorithm for a dynamic problem

SP(G) - shortest path subgraph of G has all edges a→b such that dist(a) = dist(b) + w(a, b). This is a DAG.

dist(a) is the shortest distance from vertex a to the sink vertex

CHANGED is the set of
vertices whose input or output values change.

Contd.

$||\delta||$ = $|\delta|$ + number of edges incident on some vertex in CHANGED

Bounded Incremental algorithm - for all input data-sets and for all changes that can be applied to an input data-set, the time it takes to update the output solution depends only on $||\delta||$

Deletion of one edge

Phase 1 : Find the affected vertices

1. v in deleted edge v→w
2. All vertices u such that u→v in SP(G) and outdegree_SP(G) = 0 after deleting v → w edge

Phase 2 : Determine new distances from affected vertices to sink and update SP(G)

For all a in Affected :

- dist(a) = min ({w(a, b) + dist(b)} U {inf}) s.t a → b and b not in Affected
- Push a to the priority queue iff dist(a) != inf
    -

Contd.

Update SP(G)

- Get the root vertex of the min heap (say a)
- For all b in Succ(a), if dist(a) = w(a, b) + dist(b)
  - Add a→b to SP(G)
  - Increment outdegree_SP(a)

- **Dijkstra Relaxation**

  For all c in Pred(c), if dist(c) > dist(a) + w(c, a)

  - Update dist(c) = dist(a) + w(c, a)
  - Update in the heap

Increase in edge weight

Here we can use the deletion algorithm given by Ramalingam to analyse different cases of edge weight incrementation

Do Phase 1 for all cases

Do Phase 2 only for case 4

1. Edge not in SP(G)

- Not going to affect any shortest paths
- Do nothing

2. Edge remains shortest

old:

w(v,w) + dist(w) = dist(v)

new:

w'(v,w) + dist(w) = dist(v)

- (v, w) can remain in S(G) -
- no change in shortest path structure

3. Edge becomes non-shortest

- w'(v,w) + dist(w) > dist(v)
- remove (v,w) from SP(G)
- outdegree_SP(v) > 0

Vertex v has other shortest-path edge(s) : no further work is necessary

4. Increase destroys all shortest choices of v

Only case where we do re-computation of the shortest path

- outdegree_SP(v) = 0
- v goes to WorkSet
- dist(v) has to be recomputed
- This can propagate to predecessors of v

Use the